ShelfQuery — Complete Solution

> The full build, every file. Use it to check your work, not to skip the thinking. Read the story and the hints first.

The order below is the order to build in: the deterministic guard and database first (tested with no model), then the structured SQL generator, the grounded answer, and finally the pipeline that guards every query.

Setup files

`requirements.txt`: `langchain-openai`, `openai`, `pydantic`, `python-dotenv`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here`, `LOG_LEVEL=INFO`, `DB_PATH=data/library.db`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `data/*.db`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass


class UnsafeQueryError(Exception):
    """The generated SQL was not a safe, single read-only SELECT."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError


@dataclass
class Config:
    openai_api_key: str
```

```
    log_level: str
    db_path: str


def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value


def load_config() -> Config:
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        db_path=os.getenv("DB_PATH", "data/library.db"),
    )
```

```
# src/utils/logger.py
import logging
import os


def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

MODEL_NAME = "gpt-4o-mini"
```

```
def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model=MODEL_NAME, temperature=0, api_key=config.openai_api_key
    )
```

The safety guard (deterministic, no model)

Build this first. It is the safety of the whole project, and it is plain Python, so you can prove it with hand-written SQL — no model.

```
# src/guard.py
# Decide whether a generated query is safe to run. Safe = a single,
# read-only SELECT. Everything else (writes, schema changes, chained
# statements) is rejected.
FORBIDDEN_WORDS = [
    "insert", "update", "delete", "drop", "alter",
    "create", "replace", "truncate", "attach", "pragma",
]


def is_safe_select(sql: str) -> bool:
    cleaned = sql.strip().rstrip(";").strip()
    lowered = cleaned.lower()
    if not lowered.startswith("select"):
        return False
    if ";" in cleaned:
        return False
    for word in FORBIDDEN_WORDS:
        if word in lowered:
            return False
    return True
```

The sample catalog

```
# src/db/seed.py
# Build a tiny library catalog (only if it isn't there yet): books and
# loans, so questions have real data to run against.
import os
import sqlite3

BOOKS = [
```

```
    (1, "A Wizard of Earthsea", "Le Guin", 1),
    (2, "The Dispossessed", "Le Guin", 0),
    (3, "Dune", "Herbert", 1),
    (4, "Neuromancer", "Gibson", 0),
]

LOANS = [
    (1, 2, "ada", 0),
    (2, 4, "grace", 0),
    (3, 1, "alan", 1),
]


def build_sample_db(db_path: str) -> None:
    if os.path.exists(db_path):
        return
    folder = os.path.dirname(db_path)
    if folder != "":
        os.makedirs(folder, exist_ok=True)
    connection = sqlite3.connect(db_path)
    try:
        connection.execute(
            "CREATE TABLE books ("
            "id INTEGER PRIMARY KEY, title TEXT, author TEXT, "
            "available INTEGER)"
        )
        connection.execute(
            "CREATE TABLE loans ("
            "id INTEGER PRIMARY KEY, book_id INTEGER, member TEXT, "
            "returned INTEGER)"
        )
        connection.executemany(
            "INSERT INTO books VALUES (?, ?, ?, ?)", BOOKS
        )
        connection.executemany(
            "INSERT INTO loans VALUES (?, ?, ?, ?)", LOANS
        )
        connection.commit()
    finally:
        connection.close()
```

```
# src/db/schema.py
# Return the catalog's CREATE TABLE statements as one string. The model
# needs the schema to know which tables and columns it can use.
import sqlite3
```

```
def schema_text(db_path: str) -> str:
    connection = sqlite3.connect(db_path)
    try:
        cursor = connection.execute(
            "SELECT sql FROM sqlite_master WHERE type = 'table'"
        )
        rows = cursor.fetchall()
    finally:
        connection.close()
    statements = []
    for row in rows:
        if row[0] is not None:
            statements.append(row[0])
    return "\n".join(statements)
```

```
# src/db/runner.py
# Execute a SELECT and return the rows. Only ever called after the guard
# has approved the query.
import sqlite3


def run_select(db_path: str, sql: str) -> list:
    connection = sqlite3.connect(db_path)
    try:
        cursor = connection.execute(sql)
        rows = cursor.fetchall()
    finally:
        connection.close()
    return rows
```

The SQL generator (structured output)

```
# src/schemas.py
from pydantic import BaseModel, Field


class SQLQuery(BaseModel):
    sql: str = Field(
        description="A single read-only SQL SELECT query for SQLite."
    )
```

```
# src/sql_agent.py
# Turn a plain-English question into a single read-only SELECT, using the
# schema so the model only uses real tables and columns.
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import SQLQuery

SQL_PROMPT = (
    "You write ONE read-only SQL SELECT query for SQLite that answers the "
    "question, using ONLY the tables and columns in the schema below. Do "
    "not write INSERT, UPDATE, DELETE, DROP, or more than one statement.\n\n"
    "Schema:\n{schema}\n\nQuestion: {question}"
)


def generate_sql(model, schema: str, question: str) -> str:
    prompt = ChatPromptTemplate.from_template(SQL_PROMPT)
    chain = prompt | model.with_structured_output(SQLQuery)
    result = chain.invoke({"schema": schema, "question": question})
    return result.sql
```

The grounded answer

```
# src/answer.py
# Phrase the query result in one plain sentence, using ONLY the rows.
from langchain_core.prompts import ChatPromptTemplate

ANSWER_PROMPT = (
    "Answer the question in one plain sentence using ONLY the query result "
    "rows below. If the rows are empty, say no matching data was found.\n\n"
    "Question: {question}\nRows: {rows}"
)


def summarize_answer(model, question: str, rows) -> str:
    prompt = ChatPromptTemplate.from_template(ANSWER_PROMPT)
    chain = prompt | model
    reply = chain.invoke({"question": question, "rows": str(rows)})
    return reply.content
```

The pipeline

```
# src/pipeline.py
# Read the schema, generate a SELECT, guard it, run it, and summarize the
# rows. The guard sits between generation and execution - nothing unsafe
# ever reaches the catalog.
from src.answer import summarize_answer
from src.db.runner import run_select
from src.db.schema import schema_text
from src.exceptions import UnsafeQueryError
from src.guard import is_safe_select
from src.sql_agent import generate_sql
from src.utils.logger import get_logger


def ask(model, db_path: str, question: str):
    logger = get_logger(__name__)
    schema = schema_text(db_path)
    sql = generate_sql(model, schema, question)
    if not is_safe_select(sql):
        raise UnsafeQueryError(f"Refused unsafe query: {sql}")
    logger.info(f"running: {sql}")
    rows = run_select(db_path, sql)
    answer = summarize_answer(model, question, rows)
    return answer, sql, rows
```

main.py

```
# main.py
import sys

from src.config import load_config
from src.db.seed import build_sample_db
from src.exceptions import UnsafeQueryError
from src.models.llm import get_model
from src.pipeline import ask


def main():
    if len(sys.argv) < 2:
        print('usage: python main.py "your question"')
        return
    question = sys.argv[1]

    config = load_config()
    build_sample_db(config.db_path)
```

```
    model = get_model(config)

    try:
        answer, sql, rows = ask(model, config.db_path, question)
    except UnsafeQueryError as error:
        print(f"\nRefused: {error}")
        return

    print(f"\nSQL:\n  {sql}")
    print(f"\nAnswer:\n  {answer}")


if __name__ == "__main__":
    main()
```

The unit tests (no model)

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "SHELFQUERY_TEST_SETTING"


def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")
```

```
if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_guard.py
# Run from the repo root: python -m tests.unit.test_guard
from src.guard import is_safe_select


def test_plain_select_passes() -> None:
    sql = "SELECT count(*) FROM loans WHERE returned = 0"
    result = "OK" if is_safe_select(sql) else "WRONG"
    print(f"case 1 (plain SELECT allowed)     -> {result}")


def test_writes_are_rejected() -> None:
    bad = [
        "DELETE FROM books",
        "UPDATE books SET available = 0",
        "INSERT INTO books VALUES (9, 'x', 'y', 1)",
        "DROP TABLE loans",
        "SELECT * FROM books; DROP TABLE books",
    ]
    result = "OK"
    for sql in bad:
        if is_safe_select(sql):
            result = "WRONG"
    print(f"case 2 (writes/chained rejected)  -> {result}")


if __name__ == "__main__":
    test_plain_select_passes()
    test_writes_are_rejected()
```

```
# tests/unit/test_db.py
# Run from the repo root: python -m tests.unit.test_db
import os
import tempfile

from src.db.runner import run_select
from src.db.schema import schema_text
from src.db.seed import build_sample_db
```

```
def test_seed_and_select() -> None:
    folder = tempfile.mkdtemp()
    db_path = os.path.join(folder, "test_library.db")
    build_sample_db(db_path)

    rows = run_select(
        db_path, "SELECT count(*) FROM loans WHERE returned = 0"
    )
    result = "OK"
    if rows[0][0] != 2:
        result = "WRONG"
    print(f"case 1 (2 books on loan)      -> {result}")


def test_schema_has_tables() -> None:
    folder = tempfile.mkdtemp()
    db_path = os.path.join(folder, "test_library.db")
    build_sample_db(db_path)

    schema = schema_text(db_path)
    result = "OK"
    if "books" not in schema or "loans" not in schema:
        result = "WRONG"
    print(f"case 2 (schema lists tables)  -> {result}")


if __name__ == "__main__":
    test_seed_and_select()
    test_schema_has_tables()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (plain SELECT allowed)     -> OK
case 2 (writes/chained rejected)  -> OK
```

```
case 1 (2 books on loan)      -> OK
case 2 (schema lists tables)  -> OK
```

The integration test (real model)

```
# tests/integration/test_live_ask.py
# Real model + SQLite. Run from the repo root:
#   python -m tests.integration.test_live_ask
from src.config import load_config
from src.db.seed import build_sample_db
from src.models.llm import get_model
from src.pipeline import ask


def test_counts_books_on_loan() -> None:
    config = load_config()
    build_sample_db(config.db_path)
    model = get_model(config)
    answer, sql, rows = ask(
        model, config.db_path, "How many books are currently on loan?")
    ok = "select" in sql.lower() and "2" in answer
    status = "OK" if ok else "WRONG"
    print(f"case 1 (books on loan counted) -> {status}")


def test_lists_an_author() -> None:
    config = load_config()
    build_sample_db(config.db_path)
    model = get_model(config)
    answer, sql, rows = ask(
        model, config.db_path, "Which books do we have by Le Guin?")
    ok = "earthsea" in answer.lower() or "dispossessed" in answer.lower()
    status = "OK" if ok else "WRONG"
    print(f"case 2 (author books listed)   -> {status}")


if __name__ == "__main__":
    test_counts_books_on_loan()
    test_lists_an_author()
```

Expected run:

```
$ python main.py "How many books are currently on loan?"

SQL:
  SELECT count(*) FROM loans WHERE returned = 0

Answer:
  There are 2 books currently on loan.
```

That's the whole project: the question turned into a query, the query checked by a deterministic guard before it runs, and the returned rows phrased as one plain sentence — so a librarian gets a real answer from the catalog, and no query that isn't a single read-only `SELECT` ever reaches it.